WEEK 6 · DAY 1

Backtesting: Test It on the Past First

Before you stake real money on a forecast, replay it on weeks that already happened. 🔁



NIGCOMSAT
AFRICAN ROOTED GLOBALLY POSITIONED

Publica Academy

TODAY'S MISSION · 2 HOURS

One score is a screenshot. Get the full record.



One split lies

the lucky 8 weeks



Walk forward

6 folds, refit each time



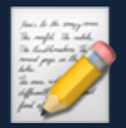
Same folds

every method, one scorer



How far ahead?

1, 4 and 8 weeks



Verdict

4 sentences

⏱️ 10 min hook · 🎤 30 min slides · 👩🏾‍💻 60 min hands-on · 🏁 20 min wrap-up

QUICK STORY

Oga Tipster's WhatsApp status 📱



One screenshot

“10 odds, **sure banker**, I don win again! ₦500 → ₦50,000. Subscribe for my VIP tips.”

one big win · no record



Your cousin the accountant

“Send me his tips for **every match last season**. I'll check them against the real results before anybody pays.”

full record · boring, honest

Replaying a method on the past, using only what was known at the time, is a **backtest**. Even the sure banker needs one. 🔁

REMEMBER WEEK 5 · DAY 2? THE LAST 8 WEEKS.

Look who won the four-baseline race 😳



A **flat average** of the training weeks scored **7.77**, better than all four code-book baselines.
Genius? Or one lucky screenshot? 📸

THE FULL RECORD: FORECAST 8 WEEKS AHEAD FROM 45 DIFFERENT WEEKS

Over many tries, the “genius” comes last



Mean MAE over 45 origins: the flat mean is **16.50**, the worst of all.

One split is one screenshot. A backtest is the full record.

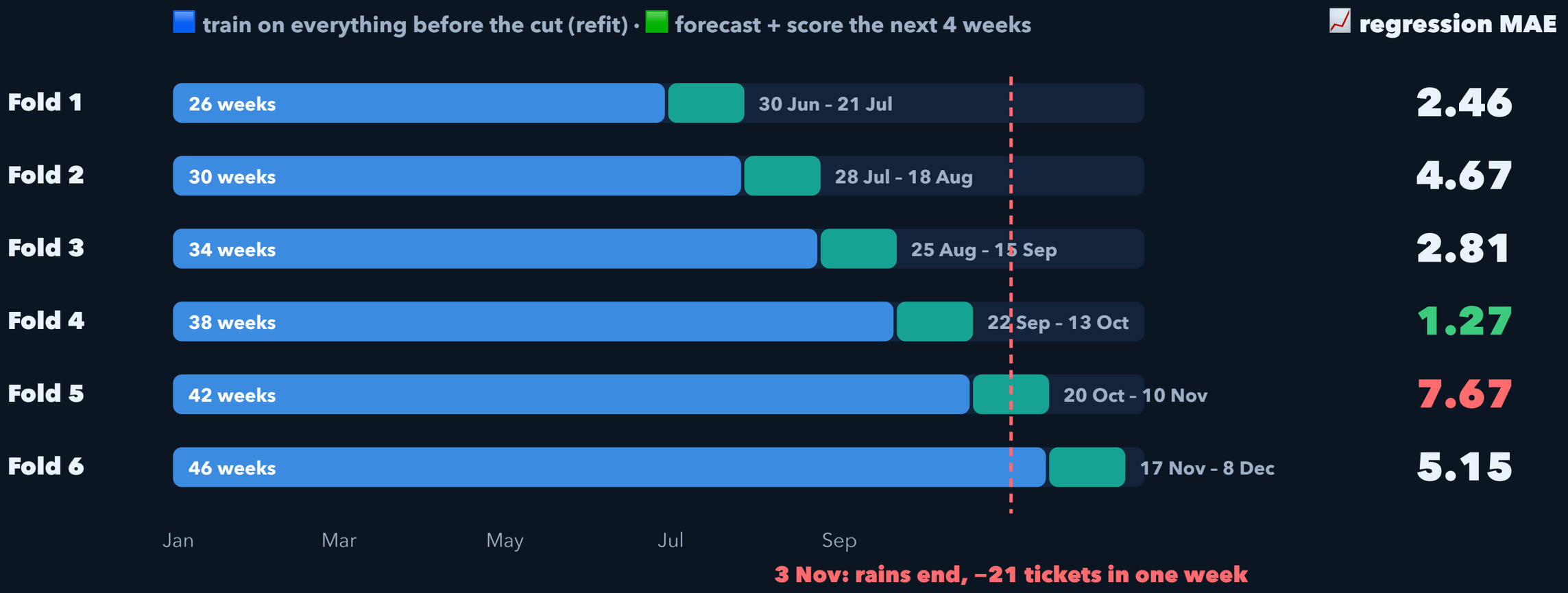
PART 1

Walk forward 

Pretend it's June. Forecast. Check. Pretend it's July…

THE CODE BOOK'S WALK-FORWARD BACKTEST · THE WEEK 5 DAY 3 REGRESSION

Six exams, each on weeks it has never seen



At every cut: **refit** on the past only → forecast the next 4 weeks → score → slide 4 weeks on.

THE WHOLE IDEA IN A FEW LINES

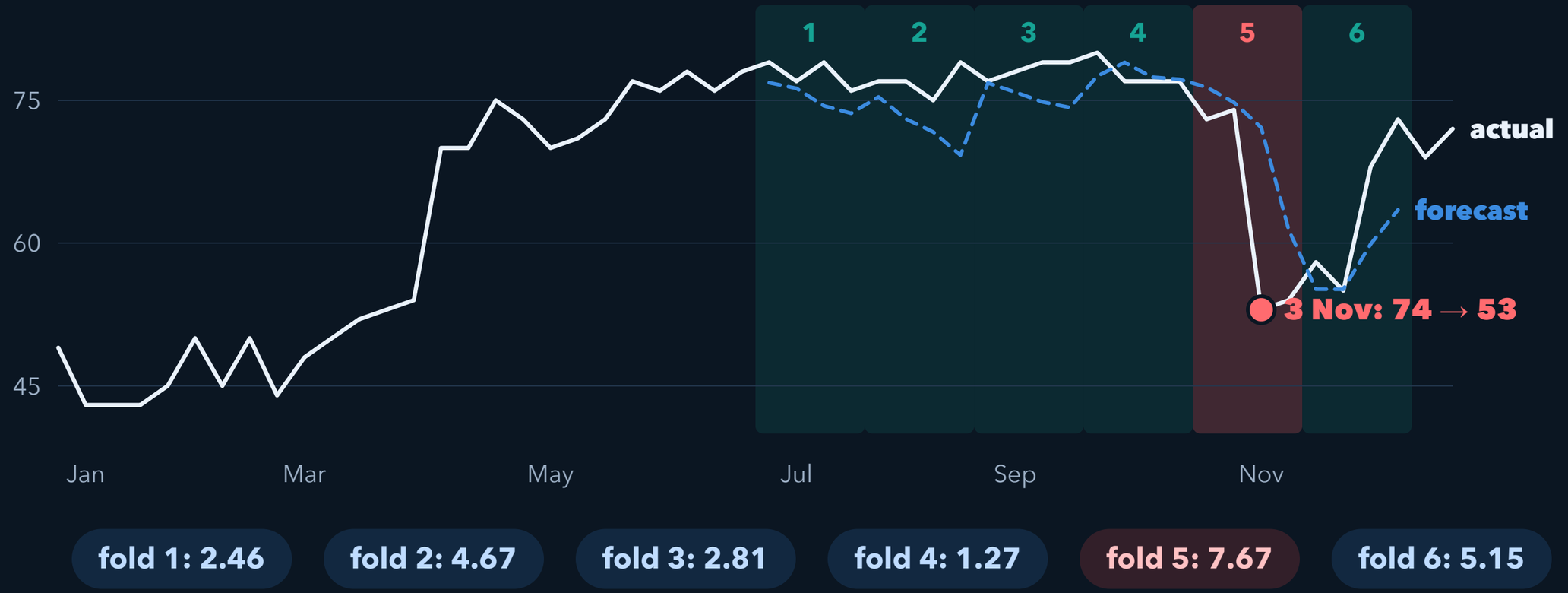
walk_forward_mae()

```python
def walk_forward_mae(df, feature_fn, x_cols, start_frac=0.5, step=4, h=4):
    feat_all = feature_fn(df)
    n = len(feat_all)
    errors = []
    for cut in range(int(n * start_frac), n - h, step):
        tr, te = feat_all.iloc[:cut], feat_all.iloc[cut:cut + h]   # past | next h weeks
        model = LinearRegression().fit(tr[x_cols], tr["target"])    # refit on the past ONLY
        errors.append(mae(model.predict(te[x_cols]), te["target"]))
    return np.array(errors)
```

`iloc[:cut]` = everything **before** the cut. `iloc[cut:cut + h]` = the next `h` weeks. Nothing from the future gets in.

SIX NUMBERS, NOT ONE

Mean 4.01… but look at fold 5



fold 1: 2.46 | fold 2: 4.67 | fold 3: 2.81 | fold 4: 1.27 | fold 5: 7.67 | fold 6: 5.15

WHAT GOES IN YOUR REPORT

Report the spread, and **when** it goes wrong

4.01

mean MAE over 6 folds

1.27

→

7.67

best → worst fold



worst = the fold with the season change

"Usually off by about 4 tickets a week, **up to 8 when the rains end**."
That sentence is worth more than any single score.

PART 2

Same folds, every method 🥊

A model score means nothing on its own.

MAE PER FOLD · LOWER IS BETTER · GREEN = BEST IN THAT FOLD

The fold-by-fold report card

FOLD	📈 REGRESSION	🧑🏾 NAIVE	📅 SEASONAL NAIVE	🧮 RATE SEASONAL NAIVE
1 · 30 Jun – 21 Jul	2.46	2.00	17.75	**1.39**
2 · 28 Jul – 18 Aug	4.67	**1.75**	18.50	3.82
3 · 25 Aug – 15 Sep	2.81	**1.00**	20.25	3.72
4 · 22 Sep – 13 Oct	1.27	**1.00**	20.00	2.49
5 · 20 Oct – 10 Nov	7.67	6.75	14.50	**2.01**
6 · 17 Nov – 8 Dec	5.15	6.25	16.00	**2.46**
Mean	**4.01**	**3.12**	**17.83**	**2.65**

Regression and naive forecast each week from last week's real count (1 week ahead). The rate forecast is made **once**, at the start of each fold, for all 4 weeks.

GRANDMA STRIKES AGAIN 👵🏾

Naive beats the regression on 5 of 6 folds

3.12
naive, mean MAE

4.01
regression, mean MAE

17.83
seasonal naive: ignores the growing network

The regression only wins the **17 Nov** fold, when the December jump arrives.
On the Week 5 Day 3 split it won. Across six folds, it doesn't.

BRING BACK THE WEEK 5 DAY 2 TRICK: TICKETS PER TERMINAL

The rate forecast, blind 4 weeks ahead

2.65

mean MAE: the lowest, with the least information

2.01

on the season-change fold (naive 6.75 · regression 7.67)

3 of 6

folds where it beats naive: honest, not a clean sweep

It knew the rains end in November: **last November's rate** showed it. ☀️

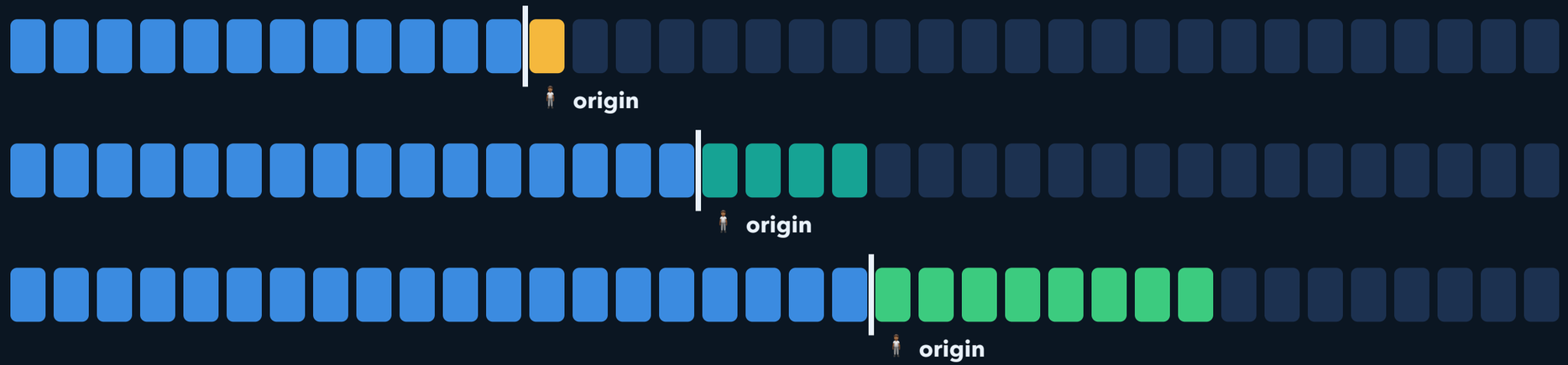
PART 3

How far ahead? 🔭

Next week's shift and next month's roster are different questions.

ROLLING ORIGIN: STAND AT EVERY WEEK

One origin per week, three horizons

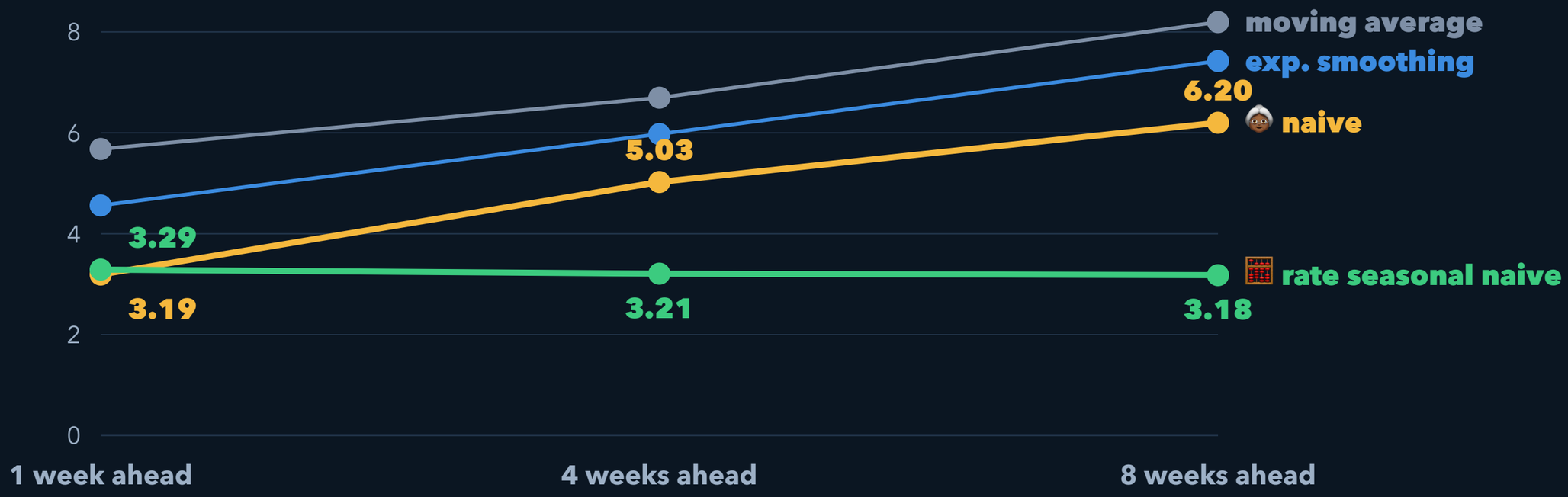


🟦 the past it may use · coloured = the weeks it forecasts blind · then step ONE week on, all year

1 week ahead	4 weeks ahead	8 weeks ahead
52 origins	49 origins	45 origins

MEAN MAE BY HORIZON · BLIND FORECASTS · EVERY ORIGIN

Grandma tires. The rate forecast doesn't. 🏃🏾



Off the chart: seasonal naive (16.1–16.2) and the flat training mean (14.8–16.5).

EVERY 8-WEEK FORECAST, ONE POINT PER ORIGIN

Naive's bad weeks: the season changes



Worst 8-week MAE: naive **18.38** · rate **5.14**. When the rains start or stop, “same as last week” is wrong for weeks on end.

THE FULL RECORD · MEAN MAE · HOW MANY ORIGINS EACH METHOD WON

Which forecast, for which question?

METHOD	1 WEEK AHEAD (52)	4 WEEKS AHEAD (49)	8 WEEKS AHEAD (45)
🧓🏾 **naive**	**3.19 · 19 wins**	5.03 · 14 wins	6.20 · 11 wins
🧱 **rate seasonal naive**	3.29 · 17 wins	**3.21 · 22 wins**	**3.18 · 21 wins**
exp. smoothing	4.56 · 6 wins	5.98 · 4 wins	7.42 · 2 wins
moving average	5.68 · 4 wins	6.70 · 6 wins	8.19 · 10 wins
seasonal naive	16.13 · 0 wins	16.10 · 0 wins	16.25 · 0 wins
flat training mean	14.79 · 6 wins	15.42 · 3 wins	16.50 · 1 wins

AND THE REGRESSION?

It can only see one week ahead



It needs last week's real count

`tickets_lag1` is a feature. To forecast week 4, you'd need week 3's real count… which hasn't happened yet.



The month-ahead roster

The NOC plans shifts 4 weeks out. A forecast that only works 1 week ahead **can't make that roster at all**.

Test at the horizon **the user actually needs**, not the one that flatters your model.

THE VERDICT, IN 4 SENTENCES

What we tell the NOC manager 📝

1. For **next week**, use naive, “same as last week” (mean error 3.2 tickets); the rate forecast ties it (3.3).

2. For the **month-ahead roster**, use last year's tickets per terminal × today's terminals (3.2 vs naive 5.0).

3. The regression doesn't earn its place: it loses to naive on 5 of 6 folds and can't forecast past one week.

4. Expect the biggest misses when the rains start or stop, so add staff margin in early April and early November.

GOLDEN RULES OF AN HONEST BACKTEST

Five rules, every time



Only the past at each origin, and **refit** every time.



Same folds, same scorer for every method, baselines included.



Report the spread and when the bad folds happen, not just the mean.



Test at the **horizon the user needs**.



Never tune on the folds you report. Fix the design (folds, step, horizon) first.

YOUR TURN · 60 MINUTES

Audit the tipster 🔍

- **A** Expose the lucky split
- **B** Walk the regression forward
- **C** Same folds for the baselines
- **D** Every origin, three horizons
- **E** Write the 4-sentence verdict



11 checks to pass

🌶️ Pepper task: refit every week. Does the verdict survive?

🏁 **Final project:** forecasting briefs must report a walk-forward spread across folds, not just the average.

TODAY IN 3 LINES

What you now know



One split is one screenshot. A backtest replays the method on many past origins.



Same folds, same scorer, every method: report the spread and when it fails.



The best method depends on the horizon. Here the simple ones win: recommend them.

Next 👉 **When will it break?** Remaining useful life, and we backtest a battery call. 🪫

Then Quiz 4 on all the forecasting days.